Lab 2 Submission

Agile Backlog Creation & Sprint Simulation in Jira

Problem Statement #36 - Restaurant Table Booking & Pre-Ordering App

Jira Project / Workspace Link: https://anshulp7470.atlassian.net/jira/software/projects/SCRUM/boards/1

GitHub: https://github.com/Ansh7-7/SE_LAB_PES1UG24CS071

Project: PES1UG24CS071_LAB2 | Jira key: SCRUM

Two sprints completed: Sprint 1 (21 pts, 16 done) and Sprint 2 (26 pts, 26 done)

1. Jira Backlog - Epics

Five Epics group the Lab 1 functional requirements into major functional themes.

Epic	Epic Name	Description
E1	**Table Selection & Reservation**	Enable diners to view a live 2D floor plan and hold, book, and manage a table reservation.
E2	**Pre-Ordering & Kitchen Prep Timeline**	Let diners attach a pre-order to a reservation and compute an estimated food-ready time for the kitchen.
E3	**Payments & Booking Confirmation**	Take a secure deposit/payment before confirming the booking and send the diner a confirmation.
E4	**Restaurant Floor Management**	Give the manager live table-status control across terminals and a view of upcoming reservations.
E5	**Platform Security & Reliability**	Cross-cutting encryption (PCI-DSS/TLS), sub-500 ms status sync, and mobile-optimised access.

2. User Stories & Story Points

Each Epic's user stories in the standard Agile format, with Fibonacci story points and priority. S3.1 was started in Sprint 1 and rolled into Sprint 2.

ID	Epic	User Story	Priority	Pts	Sprint
S1.1	E1	**As a** diner, **I want** view an interactive 2D floor plan showing which tables are open, **so that** I can see my seating options at a glance.	High	**3**	Sprint 1
S1.2	E1	**As a** diner, **I want** select an open table and place a temporary hold on it, **so that** no one else books it while I finish ordering.	High	**5**	Sprint 1
S1.3	E1	**As a** diner, **I want** book a table for a specific date, time and party size, **so that** I have a guaranteed seat when I arrive.	High	**5**	Sprint 1
S1.4	E1	**As a** diner, **I want** edit or cancel my reservation before the cutoff window, **so that** I can adjust my plans.	Medium	**3**	Backlog
S2.1	E2	**As a** diner, **I want** browse the menu and add items to a pre-order linked to my reservation, **so that** my food is prepared for my arrival.	High	**3**	Sprint 1
S2.2	E2	**As a** diner, **I want** see an estimated food-ready time calculated from my pre-order and arrival time, **so that** my meal is ready when I sit down.	High	**8**	Sprint 2
S2.3	E2	**As a** kitchen (KDS), **I want** receive queued pre-order tickets with prep timing, **so that** the kitchen can sequence preparation correctly.	Medium	**5**	Sprint 2
S3.1	E3	**As a** diner, **I want** pay a deposit or full amount through a secure payment gateway before my booking is confirmed, **so that** my reservation is guaranteed.	High	**5**	Sprint 1 -> 2
S3.2	E3	**As a** diner, **I want** apply a promo code / discount at checkout, **so that** I can reduce my bill when eligible.	Low	**3**	Backlog
S3.3	E3	**As a** diner, **I want** receive a confirmation with table number, date/time and pre-order summary via app and email/SMS, **so that** I have a record of my reservation.	Medium	**3**	Sprint 2
S4.1	E4	**As a** restaurant manager, **I want** update a table's status (Available -> Reserved -> Seated -> Billed) from any floor terminal, **so that** the floor view is always accurate.	High	**5**	Sprint 2
S4.2	E4	**As a** restaurant manager, **I want** view all upcoming reservations and their pre-orders on a dashboard, **so that** I can prepare for arrivals.	Medium	**5**	Backlog
S5.1	E5	**As a** diner, **I want** have my payment and personal data encrypted (TLS 1.2+, PCI-DSS), **so that** my information stays safe.	Medium	**8**	Backlog

ID	Epic	User Story	Priority	Pts	Sprint
S5.2	E5	**As a** restaurant manager, **I want** have table-status changes sync across all terminals in under 500 ms, **so that** overbooking never happens.	Medium	**8**	Backlog
S5.3	E5	**As a** diner, **I want** use a mobile-optimised booking flow, **so that** I can book on the go.	Low	**3**	Backlog

Sprint 1 (21 pts): S1.1, S1.2, S1.3, S2.1, S3.1. Sprint 2 (26 pts): S3.1 (carry-over), S2.2, S2.3, S3.3, S4.1.

Screenshot: Backlog with Epics, Stories & Story Points

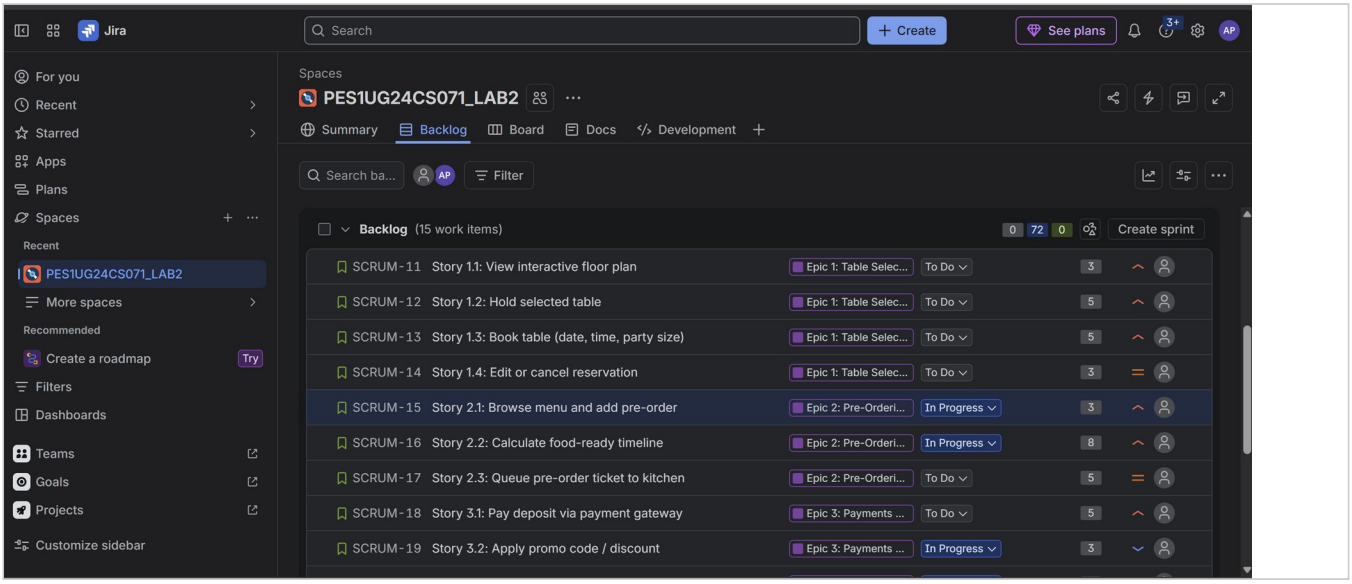


Fig. 1 - Jira Backlog with Epics, User Stories and Fibonacci story-point estimates.

3. Active Sprint Board

Sprint 1 board: four stories Done and SCRUM-18 (Pay deposit) still In Progress before completion.

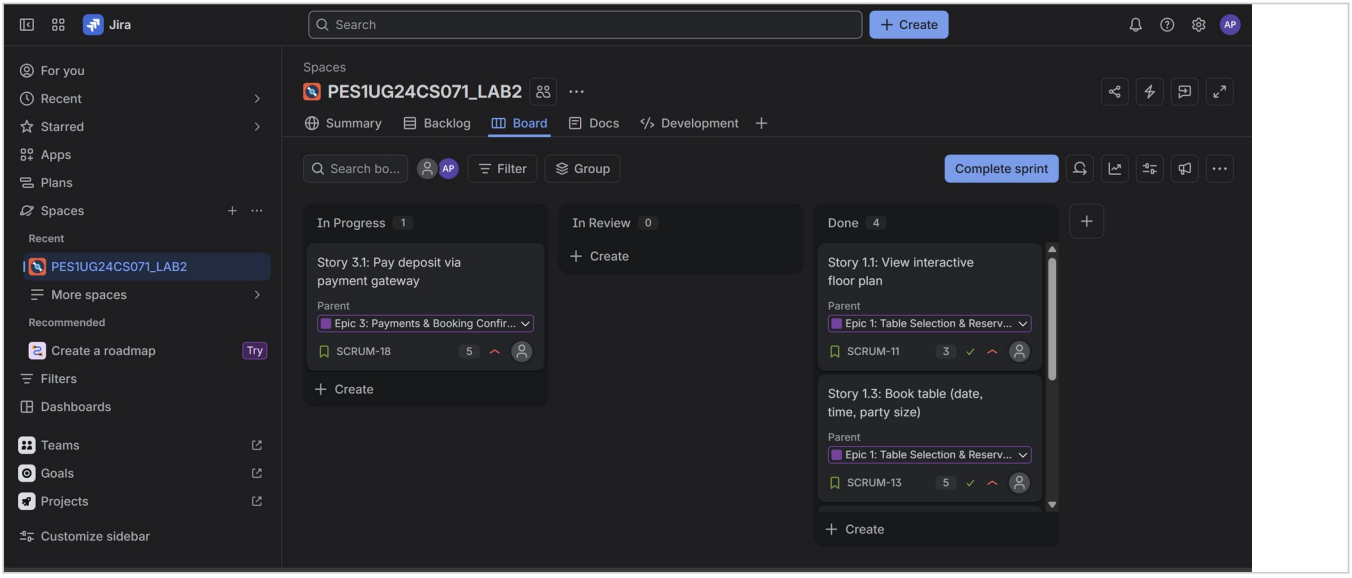


Fig. 2 - Active Sprint board (To Do / In Progress / In Review / Done).

4. Sprint Burndown Charts

Reports > Burndown Chart for each completed sprint (story points vs. time).



Fig. 3 - Sprint 1 burndown: remaining work flattens at ~5 points (SCRUM-18 rolled over).

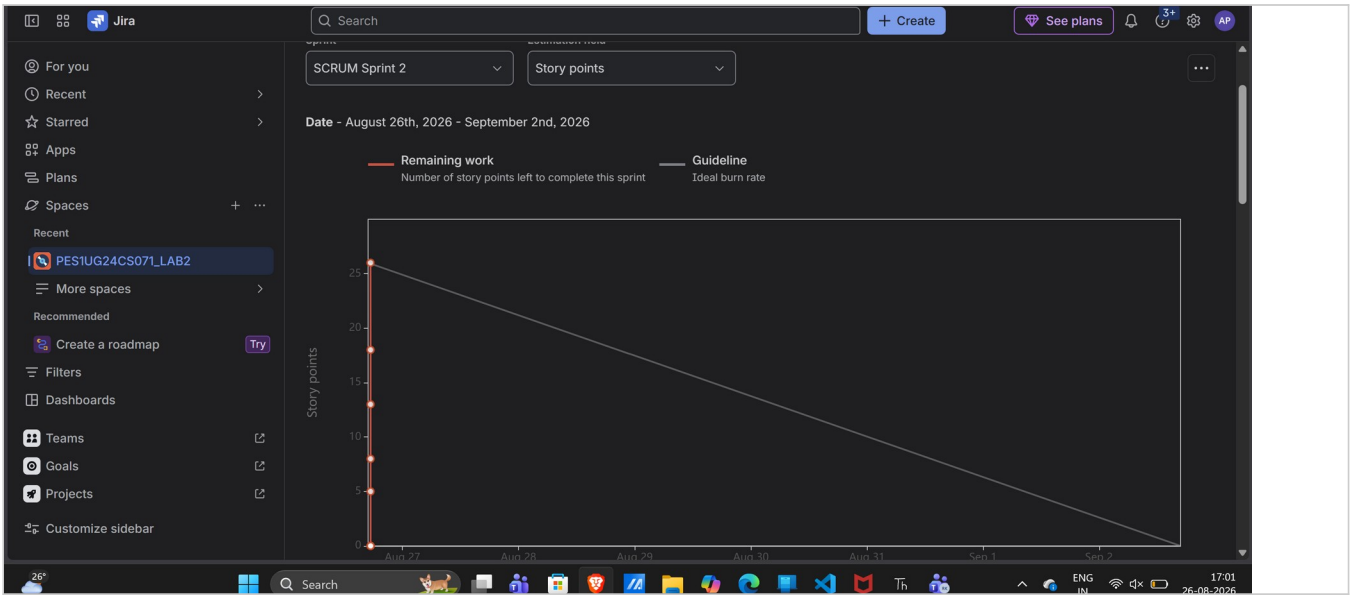


Fig. 4 - Sprint 2 burndown: remaining work burns down to 0 (26/26 completed).

5. Reflection Questions

Did the story point estimations reflect the actual effort required?

For the most part, yes. Straightforward CRUD-style stories - viewing the floor plan (S1.1), browsing the menu (S2.1) and sending the confirmation (S3.3) - were estimated at 3 points, reflecting simple read/display or single-record work. Stories that carry state or edge cases sat at 5 points: holding a table (S1.2) needs a concurrency/lock mechanism, booking (S1.3) and payment (S3.1) both involve multi-step flows with failure paths, and updating table status (S4.1) must propagate a state change reliably.

The genuinely complex work landed at 8 points: the food-ready timeline (S2.2), which aggregates per-item prep times and aligns them to the arrival slot, and the cross-cutting NFRs - data encryption/PCI-DSS (S5.1) and sub-500 ms status sync (S5.2) - which touch the whole system rather than a single record. In hindsight S2.2 is the one estimate I would revisit: it bundles the estimation algorithm and the kitchen-queue integration, and could have been split into two 5-point stories.

Because this was a solo-simulated sprint rather than a real multi-person team, there is no independent effort log to compare against, so this assessment is based on task complexity and dependency count rather than measured hours.

Was the backlog well-prioritized?

Yes - the ordering followed a clear dependency chain rather than an arbitrary ranking. Epic 1 (Table Selection & Reservation) came first because every other feature depends on a reservation existing. Epic 2 (Pre-Ordering) and Epic 3 (Payments & Confirmation) followed, since both attach to an existing reservation. Epic 4 (Floor Management) is manager-facing and only useful once bookings are flowing. Epic 5 (Platform Security & Reliability) - mostly cross-cutting NFRs - was left in the general backlog because it is validated continuously across the build rather than delivered as one discrete milestone.

Priority levels were respected in scheduling: the High-priority stories from Epics 1-3 were the ones pulled into Sprint 1 and Sprint 2, while Medium and Low stories (edit/cancel reservation, promo code, reservations dashboard, mobile flow) sat later in the sequence.

How did the simulated sprint align with the original plan?

Sprint 1 (planned 5 items / 21 points) closed with 4 of 5 stories Done (16 of 21 points, ~76%). S3.1 "Pay deposit via gateway" was still In Progress at completion and was rolled back into the backlog - standard Scrum practice when a sprint commits slightly more than one window can hold. Notably the story that slipped was a dependent, late-sequence step (payment can only be finished once the booking flow is in place), which is exactly the kind of work most likely to slip when a sprint is estimated a little optimistically.

Sprint 2 (planned 5 items / 26 points, including the rolled-over S3.1) was completed in full, closing at 26/26 points. Because this was a solo simulation rather than a team working day by day, status transitions were applied in a batch rather than emerging over the sprint's duration - so the sequence of what was prioritised to finish first is meaningful, but the timing is compressed relative to a real sprint.

What insights did the burndown chart give about team capacity?

The Sprint 1 burndown flattens out at roughly 5 points remaining instead of reaching zero, meaning actual throughput (16 of 21 points, ~76%) fell short of the commitment. That suggests real velocity is closer to 16-19 points per sprint than the 21-26 committed, and future planning should scale commitments toward that observed velocity.

Sprint 2's full 26/26 completion is an encouraging counterpoint, but with only two data points it is too early to call it a stable trend rather than variance - a third or fourth sprint would be needed to confirm. One caveat: in this simulation status changes were applied in a batch, so the burndown shows a steep single-day drop. In a real sprint that shape would be a warning sign that daily stand-ups are not surfacing incremental progress; here it is an artifact of how the simulation was run, not a genuine capacity signal.